

SIMATS ENGINEERING

ASSESSMENT TOOL 2

Name : C.Manohar

Reg No : 192372246

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA4303

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool : Data Interpretation

Weightage: 20%

QUESTIONS:

Total Marks: 20

1: HTTP Response Analysis

A web server returns the following HTTP response header:

HTTP/1.1 200 OK

Date: Mon, 01 Apr 2026 10:00:00 GMT

Content-Type: text/html

Content-Length: -150

Connection: keep-alive

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with Content-Length and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

2: DOM Structure Analysis

Consider the following DOM tree representation:

```
<html>
```

```
<head>
```

```
<title>Test Page</title>
```

```
</head>
```

```
<body>
```

```
<div>
  <p>Hello World
</div>
</body>
</html>
```

Questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph

</body>
</html>
```

Questions:

1. Identify missing closing tags.
2. Analyze issues with the tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request

5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

Code of Conduct:

I C.Manohar/192372246 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Manohar

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

ANSWERS

Question 1 – HTTP Response Analysis:

1. Incorrect/invalid fields: Content-Length: -150 is invalid. Content-Length must be a non-negative integer giving the exact byte size of the response body; a negative value violates the HTTP/1.1 specification.

2. Issue with Content-Length and its impact: A negative Content-Length gives the client no valid way to know how many bytes to read from the body. Browsers cannot parse the response correctly — the connection may hang waiting for data, the response may be truncated or rejected, and keep-alive connection reuse breaks because the client can't determine where the message ends.

3. Does the status code match the response correctness: No. 200 OK signals a successful, well-formed response, but the malformed Content-Length header makes the response internally inconsistent — the status line claims success while the header data is invalid, so the response as a whole is not correct.

4. Suggested corrections: Set Content-Length to the actual non-negative byte count of the body, e.g. Content-Length: 150. Alternatively, omit Content-Length and use Transfer-Encoding: chunked so the body length doesn't need to be pre-declared.

OUTPUT

1. HTTP RESPONSE ANALYSIS

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Analysis of Each Header Field

Header Field	Value	Meaning / Interpretation
HTTP/1.1 200 OK	200 OK	Status line. HTTP version 1.1. Status code 200 means "OK" – the request was successful and the server has returned the requested resource.
Date	Mon, 01 Apr 2026 10:00:00 GMT	The date and time when the response was generated by the server (in GMT).
Content-Type	text/html	The type of content being returned in the response body is HTML.
Content-Length	-150	The size (in bytes) of the response body. Note: Content-Length should not be negative . It may indicate a server error or misconfiguration.
Connection	keep-alive	The server will keep the connection open for further requests from the same client.

Summary

- The server has **successfully** processed the request.
- The response content is **HTML**.
- The connection will remain open for additional requests.
- There is an issue with **Content-Length** as it has a negative value (**-150**).
- This may cause problems in some clients while reading the response.

Corrected Response Header (Example)

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: 150
Connection: keep-alive
```

Key Points

- 200 OK → Success
- Content-Type tells the browser how to render the content.
- Content-Length tells the browser how much data to expect.
- keep-alive improves performance by reusing the same connection.

Conclusion: The response is mostly valid and indicates success, except Content-Length which is invalid due to a negative value.

Question 2 – DOM Structure Analysis:

1. Missing/improperly closed elements: The `<p>` element ("Hello World") is opened but never explicitly closed before the parent `<div>` closes — the `</p>` tag is missing.

2. How the browser will interpret this structure: Per the HTML5 parsing algorithm, browsers automatically close an open `<p>` element when they encounter the closing tag of an ancestor that cannot contain a `<p>` (here, `</div>`). So the browser implicitly inserts `</p>` right before `</div>`, and all major browsers behave consistently for this specific case — but the markup is still not well-formed by HTML authoring standards.

3. Impact on rendering and layout: Visually the page still renders as intended because the implicit closing matches the intended structure. However, relying on implicit tag closing is fragile — adding further nested elements after the `<p>` could produce unexpected nesting, and HTML validators will flag the document as invalid, which hurts maintainability and consistency across tools/parsers.

```
<html>
<head>
  <title>Test Page</title>
</head>
<body>
  <div>
    <p>Hello World</p>
  </div>
</body>
</html>
```

4. Corrections to make the DOM valid: Add the explicit `</p>` closing tag as shown above so every element is properly closed and nested.

OUTPUT

2. DOM STRUCTURE ANALYSIS – EXECUTION OUTPUT

1. Given HTML (With Issues)

```
1 <html>
2 <head>
3   <title>Test Page</title>
4 </head>
5 <body>
6   <div>
7     <p>Hello World
8   </div>
9 </body>
10 </html>
```

2. Missing or Improperly Closed Elements

- ✗ The `<p>` tag is not closed.
- ✗ The `<html>` tag should include the `lang` attribute (not mandatory but recommended).
- ✓ All other tags are properly closed.

3. How Browser Interprets This Structure

- Browsers are lenient and automatically fix the HTML to create a valid DOM.
- In this case:
 - The browser automatically closes the `<p>` tag before `</div>`.
 - The DOM tree will be created as if `<p>Hello World</p>` was written.

4. Impact on Rendering and Layout

- ✓ The text "Hello World" will be displayed inside the paragraph.
- ✓ Layout may look correct in most modern browsers.
- ⚠ However, relying on browser error-correction can cause:
 - Inconsistent behavior across different browsers.
 - Issues when more complex CSS or JavaScript is applied.
 - Problems in future maintenance and validation.

Visual Output in Browser

Test Page x +

file:///C:/test.html

Hello World

5. Suggested Corrections (Make the DOM Valid)

```
<html lang="en">
<head>
  <title>Test Page</title>
</head>
<body>
  <div>
    <p>Hello World</p>
  </div>
</body>
</html>
```

Changes Made:

- ✓ Added `lang="en"` to `<html>`
- ✓ Closed the `<p>` tag properly.

6. DOM Tree Created by Browser (Auto-corrected)

```
graph TD
    html[html] --> head[head]
    html --> body[body]
    head --> title[title]
    title --> titleText["Test Page"]
    body --> div[div]
    div --> p[p]
    p --> pText["Hello World"]
```

7. Summary

- ✗ The `<p>` element is not closed in the given HTML.
- Browsers automatically close the `<p>` tag before `</div>`.
- The page renders correctly, but the HTML is invalid.
- Always write valid and well-formed HTML for consistent rendering, better compatibility, and easier debugging.

Question 3 – GET vs POST in Login System:

1. Differences in data transmission: GET appends form data as a visible query string in the URL (/login?user=admin&pass=1234), which gets stored in browser history, bookmarks, and server access logs. POST sends data inside the request body (hidden from the URL), is not cached or logged by default, and has no practical size limit.

2. Which method is more suitable for login and why: POST is more suitable. It keeps credentials out of the URL, avoids browser/proxy/server caching and logging of sensitive data, and is the semantically correct method since login changes server-side session state rather than just retrieving a resource.

3. Security risks of using GET for authentication: Credentials appear in plaintext in the URL, so they are stored in browser history, saved in server and proxy access logs, may be leaked via the Referer header to third-party sites, and can be accidentally shared if a user copies or bookmarks the link.

4. Best practices for secure login implementation: Always use POST over HTTPS/TLS so the body is encrypted in transit; hash and salt passwords on the server (never store or compare in plain text); use CSRF tokens; rate-limit and lock out repeated failed login attempts; and issue secure, HttpOnly, SameSite session cookies rather than passing credentials on every request.

OUTPUT

3. GET vs POST in Login System – EXECUTION OUTPUT

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

1. Data Transmission Difference

GET Method

Browser → GET /login?user=admin&pass=1234 HTTP/1.1 → Server

Data in URL (Query String): /login?user=admin&pass=1234

POST Method

Browser → POST /login HTTP/1.1 → Server

Data in Body (Not in URL): user=admin&pass=1234

2. Which Method is More Suitable for Login and Why?

POST is more suitable for login.

- Credentials are sent in the request body, not in the URL.
- Not logged in browser history or server logs.
- Less chance of exposure.
- Supports larger data.
- Works with HTTPS for encryption.

Conclusion: POST provides better security and privacy, so it is preferred for login systems.

3. Security Risks in Using GET for Authentication

- Credentials are visible in the URL.
- Stored in browser history.
- May appear in server logs.
- Can be seen in bookmarks and shared links.
- Vulnerable to URL sniffing.
- Easily exposed if someone shares the link.

Example (Risky GET URL)

Login Page: /login?user=admin&pass=1234

Anyone who sees this URL can know the username and password.

4. Best Practices for Secure Login Implementation

- ✓ Use POST method for sending credentials.
- ✓ Always use HTTPS to encrypt data in transit.
- ✓ Validate and sanitize input on the server side.
- ✓ Use strong password policies.
- ✓ Implement rate limiting to prevent brute force attacks.
- ✓ Use session management and logout properly.
- ✓ Never store passwords in plain text.

5. Execution Demonstration

A. GET Request (Less Secure)

GET Request in Browser

http://example.com/login?user=admin&pass=1234

What is visible?

- URL contains username and password.
- Visible to anyone who sees the URL.
- Stored in history and logs.

B. POST Request (More Secure)

POST Request in Browser

http://example.com/login

What is visible?

- Only URL is visible.
- Credentials are sent in the body (hidden).
- Not stored in browser history.

Feature	GET	POST
Data Location	URL (Query String)	Request Body
Data Visibility	Visible	Hidden
Security	Low	Moderate
Browser History	Stored	Not Stored
Use in Login	Not Recommended	Recommended

Question 4 – HTML Code Inspection:

1. Missing closing tags: The <h1>Welcome heading and the <p>This is a paragraph paragraph are both opened but never explicitly closed with </h1> and </p>.

2. Issues with the tag: The tag is missing the alt attribute, which is important for accessibility (screen readers) and for graceful degradation if the image fails to load. Since is a void element, it needs no separate closing tag in HTML5, but in strict XHTML it would need to be self-closed as .

3. How browsers will handle these errors: Browsers apply HTML5's built-in error-recovery rules: they auto-close <h1> when the following <p> starts (since a heading cannot contain a paragraph), and auto-close the <p> when the parent <body> ends. The page will generally still render as intended because these implicit-closing rules are standardized, but the markup remains invalid and will fail validation.

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome</h1>
<p>This is a paragraph</p>

</body>
</html>
```

4. Corrected version: See the corrected markup above — the heading and paragraph are explicitly closed, and the tag now includes a descriptive alt attribute.

OUTPUT

4. HTML CODE INSPECTION – EXECUTION OUTPUT

Given HTML Snippet (With Line Numbers)

```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>Sample</title>
5   </head>
6   <body>
7     <h1>Welcome
8     <p>This is a paragraph
9     
10    </body>
11  </html>
```

1. Missing Closing Tags

Issue	Details
1 <h1> tag is not closed.	It should be closed with </h1> before the next element.
2 <p> tag is not closed.	It should be closed with </p> before the tag.

2. Issues with Tag

Issue	Details
1 No alt attribute	Alt text is important for accessibility and SEO.
2 Image may not exist	If "image.jpg" is missing or path is incorrect, a broken image icon will be shown.

3. How Browsers Will Handle These Errors

- Browsers are tolerant and automatically fix the code.
- The <h1> tag will be closed before <p>.
- The <p> tag will be closed before .
- The page will render, but the HTML is not valid.
- Relying on this can cause problems in complex layouts, CSS, or JavaScript.

Execution Output in Browser (Actual Rendering)

Sample

← → ↻ 🔍 file:///C:/Sample.html

Welcome

This is a paragraph

4. Corrected Version of the Code

```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <title>Sample</title>
5   </head>
6   <body>
7     <h1>Welcome</h1>
8     <p>This is a paragraph</p>
9     
10    </body>
11  </html>
```

Comparison: Before vs After

Before (Given Code)	After (Corrected Code)
<h1> and <p> not closed missing alt attribute - HTML is invalid - Browser auto-corrects the errors	- All tags properly closed has alt attribute - HTML is valid and standards-compliant - Better for accessibility, SEO, and maintenance

Key Takeaways

Point	Explanation
Valid HTML	Writing valid HTML ensures consistent rendering across all browsers.
Accessibility	Using alt attribute helps screen readers and improves accessibility.
Best Practice	Always close tags properly and validate your code.
Tools	Use tools like W3C Validator, Browser DevTools to find issues.

Conclusion:

✔ The given HTML has missing closing tags and missing alt attribute in . Browsers auto-correct the errors, but the code should be corrected for better compatibility, accessibility, and maintainability.

Question 5 – Browser-Server Communication Flow:

1. Interpreting each step: The user types a URL, which the browser parses and checks against its cache. A DNS lookup resolves the domain name to an IP address. The browser then opens a TCP connection (with a TLS handshake if HTTPS) and sends an HTTP request. The server processes the request (routing, business logic, database access) and generates a response, which travels back over the same connection. Finally, the browser parses the returned HTML/CSS/JS and renders the page.

2. Where delays can occur: Delays can occur during uncached DNS resolution, the TCP/TLS handshake, slow server-side processing (heavy queries or backend logic), network latency/limited bandwidth, and client-side parsing/rendering of large pages or render-blocking scripts and stylesheets.

3. Role of DNS in the process: DNS translates the human-readable domain name into the numeric IP address that the browser needs before it can open any network connection. If DNS resolution is slow (no local cache, distant or overloaded name servers), the entire request is delayed before the HTTP request is even sent.

4. Suggested optimizations to improve performance: Use DNS caching/prefetching, serve content via a CDN to shorten network paths, enable HTTP keep-alive/connection reuse, use HTTP/2 or HTTP/3 for multiplexed requests, add server-side caching, compress responses with gzip/Brotli, and minify/defer non-critical CSS/JS to speed up rendering.

OUTPUT

